I am going to add the reviewer agent later, for now I am thinking about how I can modify/make my agent better. Remmeber, the compition is evalated based on the folling: "Your submission will be evaluated on:

Accuracy: Does the generated SQL correctly answer the user's question?
Robustness: Can your agent handle various types of questions (aggregations, joins, filters, etc.)?
Error Handling: Does your agent gracefully handle invalid or ambiguous questions?
Code Quality: Is your code well-organized, documented, and maintainable?
Innovation: Creative approaches and novel solutions are encouraged!". What are some advanced Agentic Techniques I can incorporate that will definetely help my Text to SQL agent? For example, can I make this innovative or include Any sort of advanced NLP techniques? I have lots of time to learn so help me think of ideas

You're already in a strong spot; now you want "advanced agentic techniques" that improve **accuracy/robustness/error-handling** without turning your codebase into a research project. Below are techniques that are both *practical* and *publishable-looking* (innovation points), and they map cleanly onto the failure modes you saw (hallucinated columns like `order_total`, wrong literals like `'California'` vs `'CA'`, missing names, etc.).

1) Candidate generation + execution voting (self-consistency, but with DB)

Instead of 1 SQL, generate **K candidates** (e.g., K=3–5) with small variations (temperature/seed/prompt variants), execute them, then pick the "best" using a deterministic selector. This is a core idea in modern text-to-SQL systems (generate candidates → refine/choose).[1] [2]

How to pick the best without an LLM reviewer:

- Discard non-executing queries.
- Prefer queries that return **non-empty results** when the question expects results.
- Prefer queries whose output schema matches the question intent (e.g., question says "store name" → query selects `store_name`).
- If multiple candidates remain, choose the one with the simplest plan (EXPLAIN cost/rows heuristic) or the one that matches "expected shape" constraints.

Why it helps:

- Your failures often come from one “bad assumption” (invented total columns). Another sample often gets it right.
- This boosts robustness without needing a separate reviewer agent yet.

2) Test-driven refinement loop (mini “unit tests” for SQL)

This is an agentic technique: after generating SQL, the agent **creates a few invariants** (tests) that should hold, runs them, and repairs if violated. It’s similar in spirit to test-driven self-refinement approaches in the literature.[3] [1]

Examples of invariants (auto-derived from the question):

- “top 3 customers” → result row count should be 3 (or ≤3 if ties logic differs).
- “in January 2017” → all `order_date` must be within that range.
- “percentage of total revenue” → percentages should sum to ~100 (allow small floating error).
- “each customer” → group by `customer_id` (or at least ensure uniqueness).

Implementation idea:

- Add a lightweight `derive_assertions(question)` that returns a few checks.
- Run them after execution; if fail, regenerate with the failed assertion messages.

This is **highly innovative** for a competition submission because it looks like “self-debugging with formal-ish constraints,” and it directly improves “robustness” + “error handling.”

3) Column exploration / value probing (schema + value grounding)

Your “California vs CA” miss is a **value grounding** problem. A strong technique is to do a quick DB probe when the user mentions a value that likely comes from a categorical column (state, city, brand_name, category_name). ReFoRCE-style agents emphasize column/value exploration with execution feedback.[1]

Heuristic approach (cheap, effective):

- If question contains a likely entity (e.g., “California”, “Trek”, “Electra”, “Baldwin Bikes”), run a small query to see how it appears in the DB:
  - `SELECT DISTINCT state FROM customers LIMIT 50;`
  - `SELECT DISTINCT state FROM stores LIMIT 50;`
- Then rewrite the prompt to the generator: “Use state codes; CA not California” if that’s what you find.

This single trick fixes a surprising number of “0 rows but executes” issues.

4) “No hallucinated columns” hard guardrail (static + DB-based)

You already catch runtime errors, but you can reduce them earlier by enforcing constraints:

- Parse produced SQL for `alias.column` references.
- Resolve aliases → base tables from `FROM/JOIN`.
- Reject if any referenced column is not in schema; regenerate with a targeted message (“Column order_total does not exist in orders; compute totals via order_items quantity*list_price*(1-discount)”).

This is basically “format restriction / schema linking enforcement.” It’s not heavy NLP; it’s simple parsing + schema lookup—high ROI.

5) CTE-by-CTE execution validation (localized debugging)

A very strong agentic technique from modern systems is to force the model to write the query as CTEs, then validate each CTE step (execute partials) and repair locally.[1]

How it works in your context:

- Ask the model: “Write SQL using WITH clauses: base tables → joined facts → final aggregation.”
- After generation, split out each CTE, run `SELECT * FROM cte_name LIMIT 5` in order.
- If a CTE returns 0 rows unexpectedly or errors, you repair just that step.

Benefits:

- Fixes complex hard queries faster.
- Makes debugging deterministic and impressive (innovation + robustness).

6) Query shape planning (structured “SQL plan JSON” → SQL)

This is “agentic planning” without multi-agent overhead:

1. LLM produces a **structured plan**:

```
{
  "tables": ["orders","order_items","customers"],
  "joins": [{"left":"orders.customer_id","right":"customers.customer_id"}],
  "filters": [{"col":"orders.order_date","op":"between","value":["2017-01-01","2017-01-31
  "group_by": ["customers.customer_id"],
  "select": [{"expr":"SUM(order_items.quantity*order_items.list_price*(1-order_items.disc
  "order_by": [{"expr":"total_spent","dir":"DESC"}],
  "limit": 3
}
```

2. Your code deterministically converts that to DuckDB SQL.

Why it helps:

- Reduces “model invents columns” because the plan references explicit columns.

- You can validate the plan against schema before SQL generation (great for error handling).
- This is innovative and code-quality friendly.

7) Ambiguity handling that scores points (and avoids wrong SQL)

Evaluation includes "gracefully handle ambiguous questions." Add an ambiguity detector:

- If question lacks key constraints (e.g., "best customers" without metric; "recent orders" without date), return a **safe default** query (e.g., last 30 days if dates exist) *or* return a **clarifying question** (but careful: competition expects SQL string only—so instead you can return a conservative query and log ambiguity internally).

Practical approach:

- Define a small set of ambiguity patterns and fallback behaviors.
- Example: "best-selling products" → interpret as highest quantity sold (order_items.quantity) unless specified revenue.

This boosts robustness and error handling without extra LLM calls.

8) Failure memory (local "repair cookbook")

Every time your validator fails (binder error, unknown column, empty results), store a small record:

- question features, error type, fix applied, final working SQL pattern.
  Then at runtime, retrieve the most similar failure and inject a short "repair hint" into the prompt.

This is a lightweight, competition-friendly version of "experience replay." SQLFixAgent-like systems emphasize repair and consistency to improve semantic correctness.[4]

9) Model specialization: "two temperature modes"

Use deterministic low-temp for final output, but higher-temp for candidate diversity:

- Candidate generation: temps [0.2, 0.5, 0.8]
- Final selection: choose best executing candidate (and optionally run one refinement pass).

This makes your system more robust across question types (joins vs aggregations) with minimal code.

10) Small but powerful NLP additions (worth it, not overkill)

If you want "advanced NLP techniques" that actually help:

- **Entity normalization**: map state names → codes ("California"→"CA") using a tiny built-in dictionary plus DB probes (Section 3).
- **Numeric/temporal parsing**: detect "last month", "in 2017", "Q1 2018" and convert to explicit date ranges.

- **Intent classification** (tiny rules): detect if user asks for count vs list vs top-k vs distribution; this helps set query "shape" defaults.

These improve robustness and reduce weird outputs like selecting `store_id` when asked for store names.

What I'd implement first (highest ROI)

If you have lots of time, you still want to sequence by payoff:

1. **Candidate generation + execution voting** (big accuracy/robustness jump)[2]
2. **Value grounding probes** (fixes "0 rows" silent failures; improves semantic accuracy)[1]
3. **No-hallucinated-columns guardrail** (cuts binder errors dramatically)
4. **CTE-by-CTE validation** for hard queries (innovation + real robustness)[1]
5. **Structured plan JSON → SQL** (most innovative; also best for maintainability)

Concretely: what should "innovation" look like in your README?

You can describe your approach as:

- "Execution-guided self-consistency with candidate selection"[2]
- "Value-grounded schema linking via on-the-fly DB probing"[1]
- "CTE-localized debugging with iterative refinement"[1]
- "Rule-based ambiguity handling + safe defaults for robustness"

These sound research-y but are very implementable.

If you tell me your current constraints (max allowed LLM calls per query, latency budget, and whether evaluation penalizes slower agents), I can recommend exact settings: K candidates, retry counts, and which checks to enable by default.

⁂

1. https://arxiv.org/html/2502.00675v2
2. https://arxiv.org/html/2411.08599v1
3. https://aclanthology.org/2025.findings-emnlp.156.pdf
4. https://arxiv.org/html/2406.13408v6
5. https://arxiv.org/html/2406.13408v1
6. https://arxiv.org/html/2507.04701v1
7. https://github.com/Snowflake-Labs/ReFoRCE
8. https://www.themoonlight.io/fr/review/sqlfixagent-towards-semantic-accurate-text-to-sql-parsing-via-consistency-enhanced-multi-agent-collaboration
9. https://contextual.ai/blog/open-sourcing-the-best-local-text-to-sql-system/

10. https://www.reddit.com/r/ChatGPTCoding/comments/1jw0us4/building_a_robust_texttosql_agent_powered_by_gpt/
11. https://hao-ai-lab.github.io/blogs/reforce/
12. https://openreview.net/pdf?id=OuFlfDBwQd
13. https://arxiv.org/html/2502.00675v1
14. https://bohrium.dp.tech/paper/arxiv/2406.13408
15. https://www.emergentmind.com/topics/agentic-text-to-sql-systems